

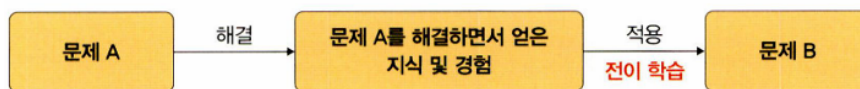
6 합성곱 신경망 II

5.3 전이 학습

전이 학습(transfer learning)이란?

이미지넷 (imageNet) 처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

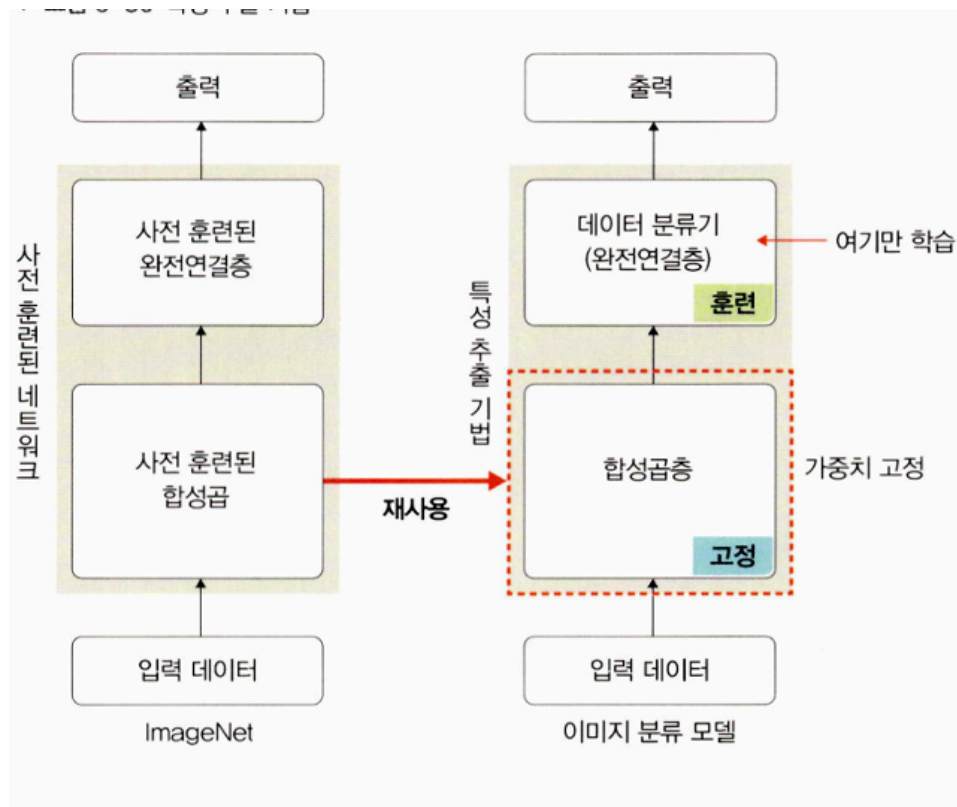
▼ 그림 5-29 전이 학습



5.3.1 특성 추출(feature extractor) 기법

데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만들.

- 합성곱층 : 합성곱층 + 풀링층
- 데이터 분류기(완전연결층) : 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스 분류



▼ 이미지 데이터 전처리 방법 정의

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
])

train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

출력

385

- torchvision.transform
 - Resize : 이미지의 크기를 조정
 - RandomResizedCrop : 이미지를 랜덤한 비율로 자른 후 데이터 크기 조정
 - RandomHorizontalFlip : 이미지를 랜덤하게 수평으로 뒤집기
 - ToTensor : 이미지 데이터를 텐서로 변환
- datasets.ImageFolder : 데이터로더가 데이터를 불러올 대상 / 방법 정의
 - data_path
 - transform : 이미지 데이터에 대한 전처리

- DataLoader

- batch_size : 한 번에 불러올 데이터양을 결정하는 배치 크기
- num_workers : 데이터를 불러올 때 사용할 하위 프로세스 개수
- shuffle : 데이터를 무작위로 섞을지 결정

- ▼ 학습에 사용될 이미지 출력

#학습에 필요한 이미지 출력

```
samples, labels = next(iter(train_loader))
classes = {0:'cat', 1:'dog'}
fig = plt.figure(figsize=(16, 24))
for i in range(24):
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()])
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- 반복자 효과 사용을 위한 iter() 과 next()

iter() 는 전달된 데이터의 반복자를 꺼내 반환하고 next() 는 그 반복자가 다음에 출력해야 할 요소 반환

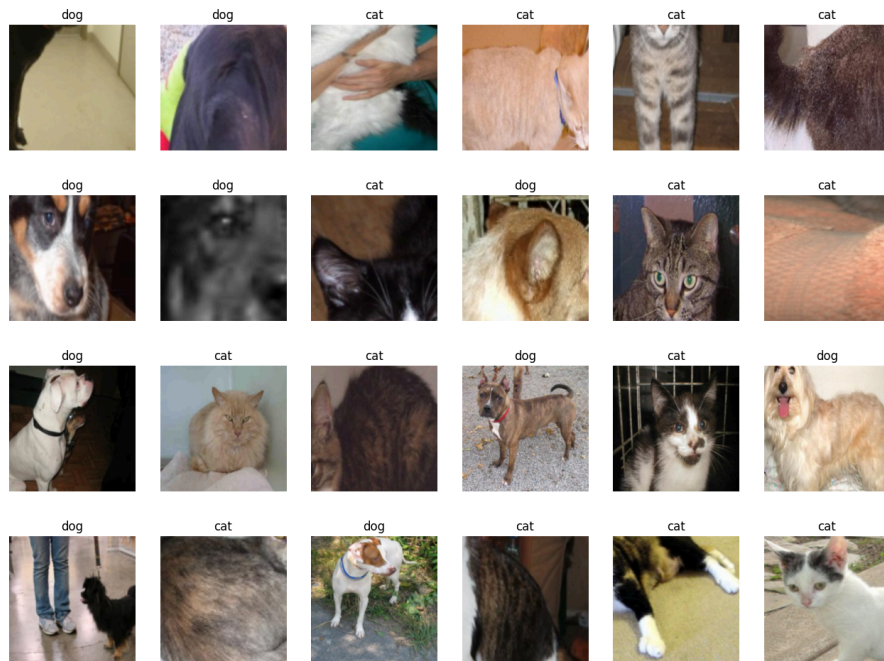
★ 책에 써있는 대로 iter(train_loader).next() 로 하면 오류가 난다. ⇒

- np.transpose() : 행과 열을 바꿈으로써 행렬의 차원을 바꾸어 줌.

- pytorch 이미지 형식 : [Channel, Height, Width] → [C, H, W]
- matplotlib.pyplot.imshow() 이미지 형식 : [Height, Width, Channel] → [H, W, C]

이미지 형식이 다르기 때문에 이를 맞추어주기 위해 transpose 를 사용.

결과



▼ 사전 훈련된 모델 resnet18

#사전 훈련된 모델 내려받기

```
resnet18 = models.resnet18(pretrained=True)
```

#사전 훈련된 모델의 파라미터 학습 유무 지정

```
def set_parameter_requires_grad(model, feature_extracting=True):
```

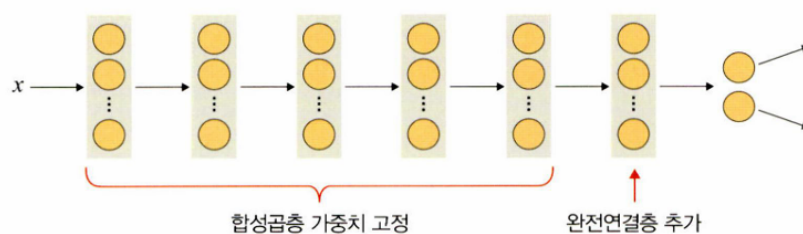
```
    if feature_extracting:
```

```
        for param in model.parameters():
```

```
            param.requires_grad = False
```

```
            #역전파 중 파라미터들에 대한 변화를 계산할 필요 X
```

```
set_parameter_requires_grad(resnet18)
```



- 합성곱층/풀링층의 가중치는 고정하고 완전연결층만 추가하게 됨.

📁 ResNet18

50 개의 계층으로 구성된 합성곱 신경망. ImageNet 데이터베이스의 100만 개 이상의 영상을 이용하여 훈련된 모델. 하지만 입력 제약이 매우 크고 충분한 메모리(RAM) 가 없으면 학습 속도가 느릴 수 있다.

사전 훈련된 모델

```
#pytorch 가 무작위의 가중치로 구성한 모델
import torchvision.models as models
resnet18 = models.resnet18()
...
squeezenet = models.squeezenet1_0()
...

#사전 학습된 모델의 가중치 값 사용하기
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
...
```

▼ 완전연결층 추가, 파라미터 확인

```
#resnet18 에 완전연결층 추가
resnet18.fc = nn.Linear(512,2) #클래스 = 2

#모델의 파라미터 값 확인
for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)
```

결과

```
fc.weight tensor([[ 0.0290, -0.0368, -0.0243, ..., 0.0268, 0.0337, 0.0388],
                  [ 0.0173, -0.0082, 0.0215, ..., -0.0272, -0.0227, -0.0236]])
fc.bias tensor([-0.0427, -0.0383])
```

▼ 모델 객체 생성 및 손실 함수 정의

```
model = models.resnet18(pretrained=True)

for param in model.parameters(): #합성곱층 가중치 고정
```

```

param.requires_grad = False

model.fc = torch.nn.Linear(512,2) #완전연결층은 학습
for param in model.fc.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() #손실 함수 정의
print(model)

```

▼ 모델 학습을 위한 함수 생성

```

def train_model(model, dataloaders, criterion, optimizer, device, num_epochs):
    since = time.time() #컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad()

            outputs = model(inputs) #순전파 학습
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward()
            optimizer.step()

```

```

        running_loss += loss.item() * inputs.size(0)
        #출력 결과와 레이블의 오차 계산 결과 누적 저장
        running_corrects += torch.sum(preds == labels.data)
        #출력 결과와 레이블이 동일한지 확인 결과 누적 저장
    epoch_loss = running_loss / len(dataloaders.dataset)
    epoch_acc = running_corrects.double() / len(dataloaders.dataset)

    print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc

    acc_history.append(epoch_acc.item())
    loss_history.append(epoch_loss)
    torch.save( model.state_dict(), os.path.join(save_dir, '{0:0=2d}.pth'.format(epoch)))
    print()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history, loss_history #반환

```

▼ 파라미터 학습 결과를 옵티마이저에 전달

```

#파라미터 학습 결과를 옵티마이저에 전달

params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
        print('\t', name)

optimizer = optim.Adam(params_to_update)

```

- Adam 옵티마이저를 사용

▼ 모델 학습

```

evice = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, crit

```

결과

```

Training complete in 10m 6s
Best Acc: 0.945455

```

▼ 테스트 데이터를 이용해서 모델 정확도 측정

```

#평가 함수
def eval_model(model, dataloaders, device):
    since = time.time() #컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('/content/catanddog/models/*.pth')
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders:
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad():

```



```

        outputs = model(inputs)

        _, preds = torch.max(outputs.data, 1)
        preds[preds>=0.5] = 1
        preds[preds<0.5] = 0
        running_corrects += preds.eq(labels.cpu()).int().sum()

    epoch_acc = running_corrects.double() / len(dataloaders.dataset)
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc
        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 6
    print('Best Acc: {:.4f}'.format(best_acc))

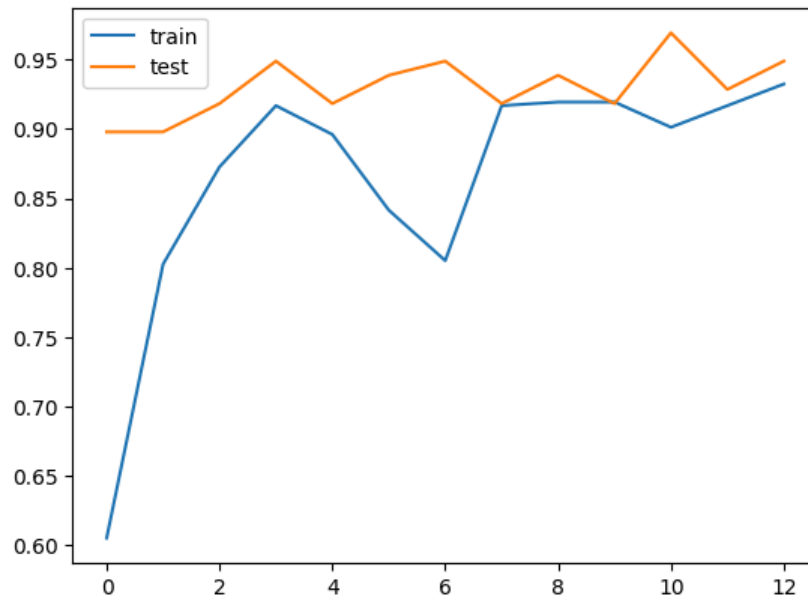
    return acc_history

#평가 진행
val_acc_hist = eval_model(resnet18, test_loader, device)

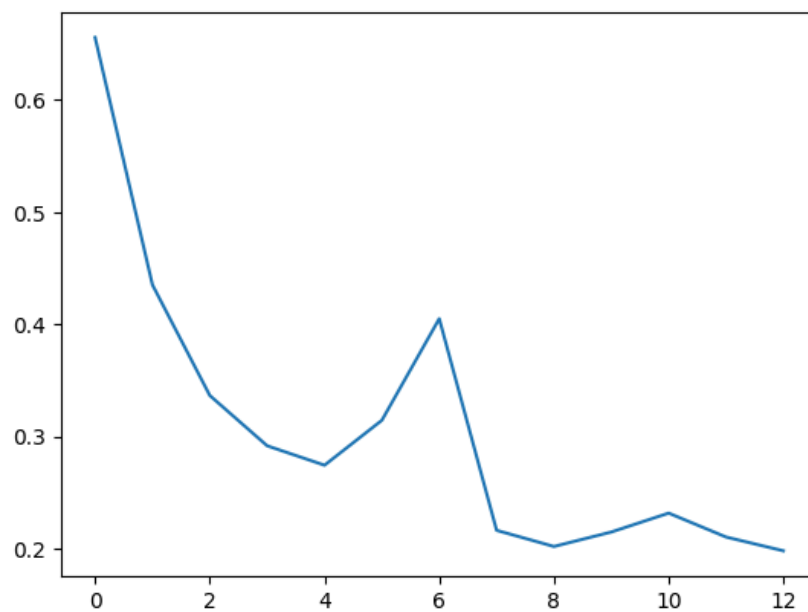
```

결과

- ▼ 학습 결과 시각적으로 살펴보기
 - 훈련/테스트 데이터의 정확도



- 훈련 데이터의 오차



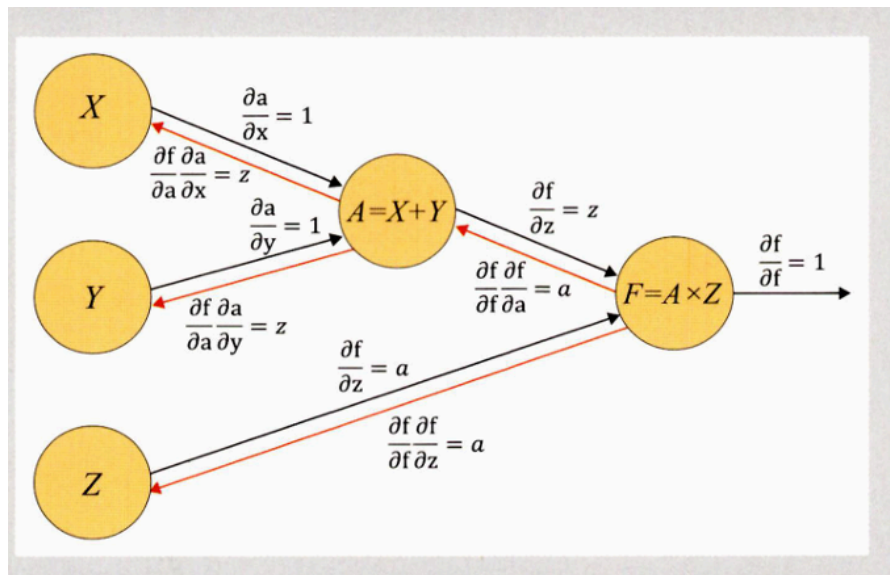
▼ 예측 데이터 출력해보기

```
#전처리 함수
def im_convert(tensor):
    image = tensor.clone().detach().numpy()
    image = image.transpose(1,2,0)
    image = image * (np.array((0.5, 0.5, 0.5)) + np.array((0.5, 0.5, 0.5)))
    image = image.clip(0, 1)
    return image
```

- `tensor.clone()` : 기존 텐서의 내용을 복사한 텐서 생성
- `.detach()` : 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에는 영향을 주지 않는다는 의미

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

📌 계산 그래프(computational graph) : 계산 과정을 그래프로 나타낸 것. 여러 개의 노드와 그 노드들을 연결하는 에지로 구성.



- 국소적 계산을 가능하게 하고, 역전파를 통한 미분 계산이 편리하다.
- `clip()` : 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용.

#예측 결과 출력

```
classes = {0:'cat', 1:'dog'}
```

```
dataiter = iter(test_loader)
```

```
images, labels = next(dataiter)
```

```
output = model(images)
```

```
_, preds = torch.max(output, 1)
```

```
fig = plt.figure(figsize=(25, 4))
```

```
for idx in np.arange(20):
```

```
    ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])
```

```
    plt.imshow(im_convert(images[idx]))
```

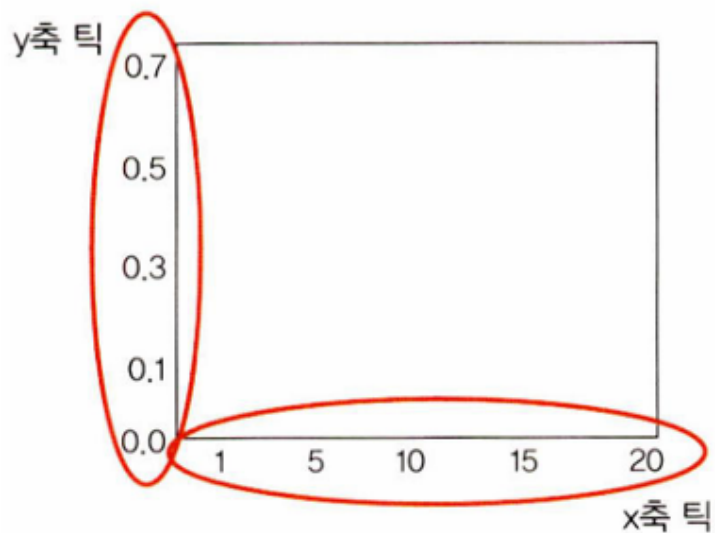
```
    ax.set_title(classes[labels[idx].item()])
```

```
    ax.set_title("{}{}".format(str(classes[preds[idx].item()]), str(classes[la
```

```
plt.show()
```

```
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- add_subplot(행, 열, 인덱스, 틱) : 한 화면에 여러 개의 이미지를 담기 위해 사용.
 - 틱



- classes[preds[idx].item()] : preds[idx].item() 값이 0 과 1 중 어떤 값인지 판별하겠다는 의미

결과



5.3.2 미세 조정 기법

사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식. 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치 일부를 재학습시키는 것.

- 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우

→ 모델 전체를 재학습

- 데이터셋이 크고 사전 훈련된 모델과 유사성 클 경우

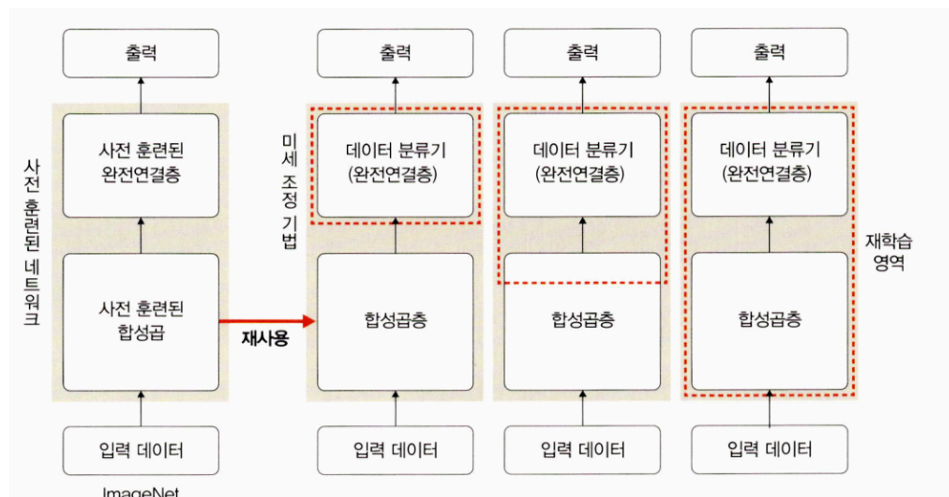
→ 합성곱층의 뒷부분과 데이터 분류기를 학습시킴. 데이터셋이 유사하기 때문에 전체를 학습시키는 것보다는 강한 특징이 나타나는 합성곱층의 뒷부분과 데이터 분류기만 새로 학습하는 것이 유리하다.

- 데이터셋이 작고 사전 훈련된 모델과 유사성 작은 경우

→ 합성곱층의 일부분과 데이터 분류기 학습. 일부 계층에 미세 조정 기법을 사용해도 효과가 없을 수 있기 때문

- 데이터셋이 작고 사전 훈련된 모델과 유사성 큰 경우

→ 데이터 분류기만 학습. 많은 계층에 미세 조정 기법을 사용하면 과적합 발생 가능성



5.4 설명 가능한 CNN

- 설명 가능한 CNN (explainable CNN) : 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

5.4.1 특성 맵 시각화

- PIL(Python Image Library) 사용 : 이미지 분석 및 처리를 쉽게 할 수 있도록 도와주는 라이브러리

```
pip install pillow
```

▼ 설명 가능한 네트워크 생성

#13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), #inplace=True 는 기존의 데이터를 연산의 결과값으로
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
```

```

nn.ReLU(inplace=True),
nn.Dropout(0,4),
nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0,4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0,4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),

nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0,4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0,4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
)
self.classifier = nn.Sequential(
    nn.Linear(512,512, bias=False),
    nn.Dropout(0,5),
    nn.BatchNorm1d(512),
    nn.ReLU(inplace=True),

```

```

nn.Dropout(0.5),
nn.Linear(512, num_classes)
)
def forward(self,x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x)

```

- 13개의 합성곱 계층 (Conv2d)

★ 각 계층마다 다음 과정 반복

[Conv2d] → [BatchNorm2d] → [ReLU] → (Dropout) → (MaxPool2d)

★ 5번의 채널 증가

3 → (64 - 64) → (128 - 128) → (256 - 256 - 256) → (512 - 512 - 512 - 512 - 512 - 512)

채널 증가 = 특징 맵(feature map) 의 개수 증가

⇒ 이미지 한 장 → 여러 장의 필터

★ 채널을 키우는 이유

- 더 많은 특징 학습 가능

채널 수 = 학습하는 필터 수 = 동시에 학습할 수 있는 패턴 수

- 공간 해상도를 줄이면서 정보량 유지

MaxPool 과정을 통해 너비/높이 감소

⇒ 채널 수를 늘려서 **정보 손실 보완**

- 복잡한 특징 학습을 위해 필요

★ 같은 채널 수에서 Conv2d 를 여러번 수행하는 이유

- feature map 의 공간 해상도가 줄어든 상태에서 심층 특징 을 정교하게 학습해야 하므로 채널 수는 그대로 두고 다양한 필터로 같은 분석을 반복하는 구조

- 2개의 완전연결층

CNN 의 마지막 출력 벡터를 두 단계에 거쳐 고차원(512, 512) → 중간 차원 → 클래스 수(512, 2) 로 변환하는 것

- 512차원 특징에서 RELU 를 통해 **비선형 변환**을 한번 더 수행

⇒ CNN에서 학습한 대상의 특징으로 판단 근거를 확립하는 과정이라고 생각할 수 있음.

- 로그 소프트맥스 (log_softmax()) 함수

일반 소프트맥스 함수는 기울기 소멸 문제가 있기 때문에 소프트맥스의 결과값에 log 값을 취한 연산

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$
$$\text{logsoftmax} = \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right)$$
$$= x_i - \log\left(\sum_{j=1} e^{x_j}\right)$$

▼ 모델 객체화

#모델 객체화

model = XAI() #XAI 클래스의 인스턴스 생성

model.to(device) #모델의 모든 파라미터를 GPU 또는 CPU 로 옮기는 가정

model.eval() #모델을 평가 모드로 전환

- 결과

```
XAI(  
  (features): Sequential(  
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)  
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): Dropout(p=0, inplace=3)  
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (6): ReLU(inplace=True)  
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (10): ReLU(inplace=True)
```

- 평가 모드

Dropout, BatchNorm 은 학습할 때와 평가할 때 동작 방식이 다르다.

Dropout	일부 뉴런 랜덤 제거	전부 사용 (비활성화)
BatchNorm	배치별 평균/분산 사용	학습된 평균/분산 고정 사용

▼ 특성 맵을 확인하기 위한 클래스 정의

```
#특성 맵을 확인하기 위한 클래스 정의

class LayerActivations():
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

        #self.features 에 특정 레이어의 출력(특성맵) 저장
    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self):
        self.hook.remove()
```

- hook 기능 : print 문 처럼 각 계층의 활성화 함수 및 기울기 값 확인. 주로 중간 결과값들을 출력하기 위해 사용된다.
- register_forward_hook(self.hook_fn) : 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오기. forward 연산이 실행될 때 자동으로 hook_fn 을 호출

▼ 이미지 호출

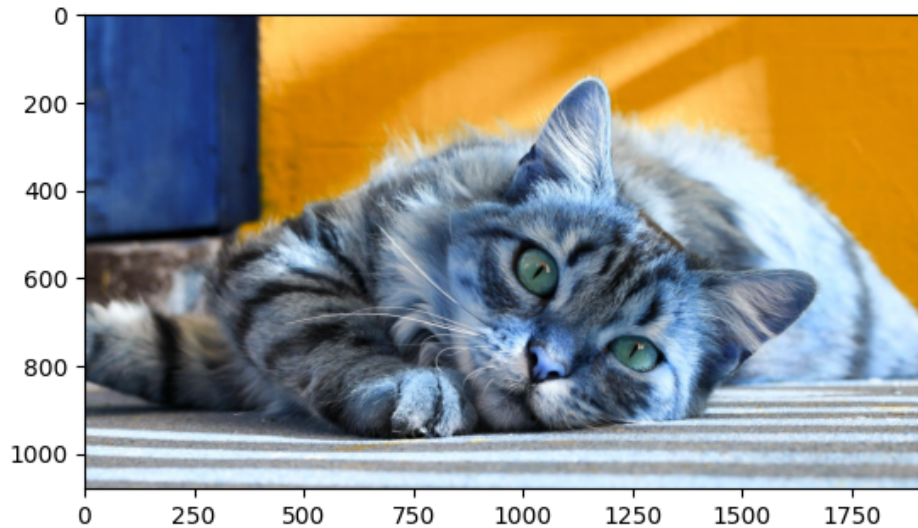
```
#이미지 호출

img = cv2.imread('/content/cat.jpg')
plt.imshow(img)

#이미지 크기 변경
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
print(img.shape)
```

- `unsqueeze()` : 1차원 데이터를 생성하는 함수로 이미지 데이터를 텐서로 변환하고 그 변환한 데이터를 1차원으로 변경하겠다는 의미.

호출된 이미지



▼ Conv2d 특성 맵 확인

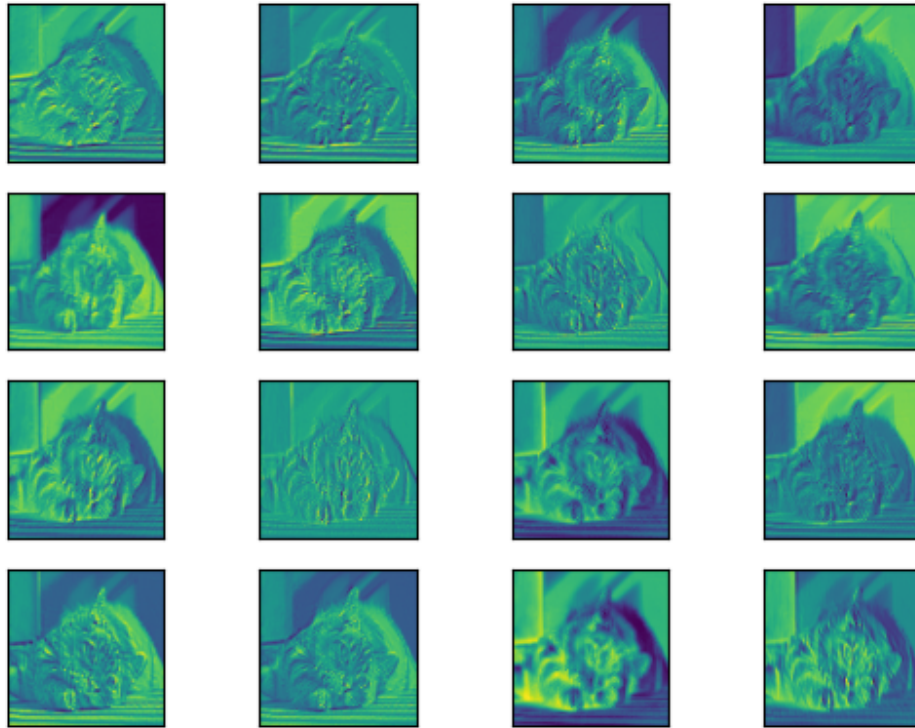
#Conv2d 특성 맵 확인

```
result = LayerActivations(model.features, 0)
model(img)
activations = result.features

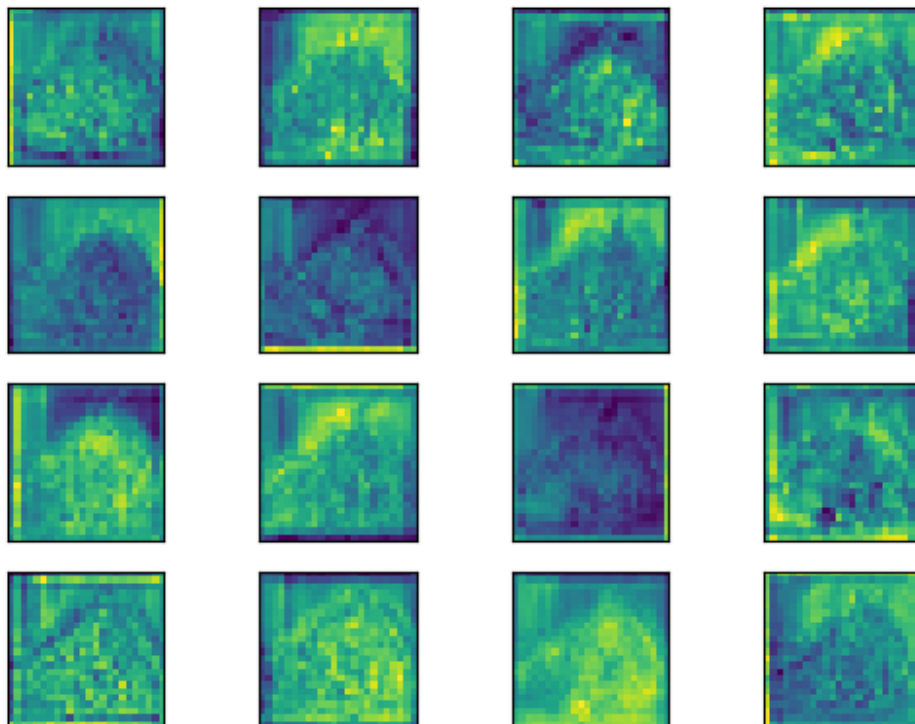
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, bottom=0, right=1, top=1, hspace=0.05, wspace=0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

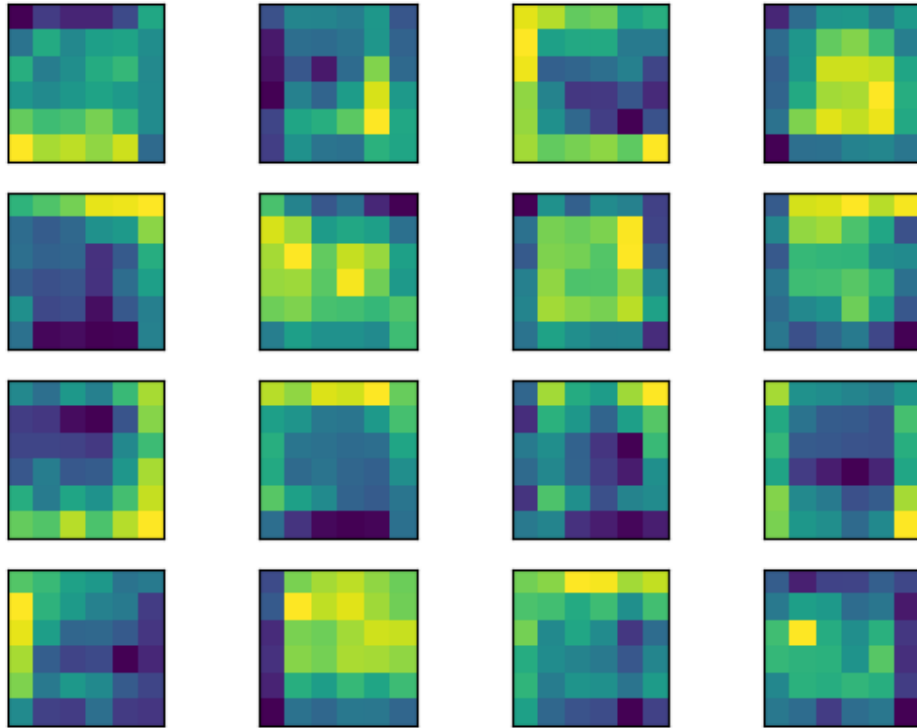
첫 번째 계층 결과



20번째 계층 결과



40번째 계층 결과

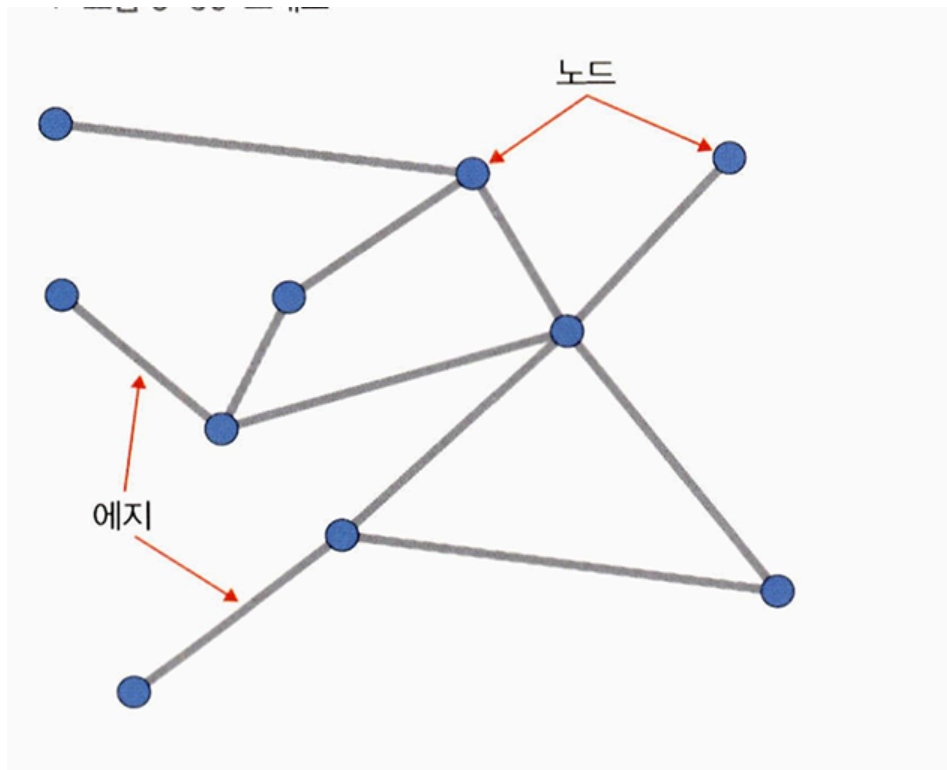


⇒ 계층이 깊어질수록 원래 입력 이미지에 대한 형태는 찾아볼 수 없다. 출력층에 가까울수록 원래 형태보다는 이미지의 특징들만 전달되는 것을 확인 가능.

5.5 그래프 합성곱 네트워크

그래프란?

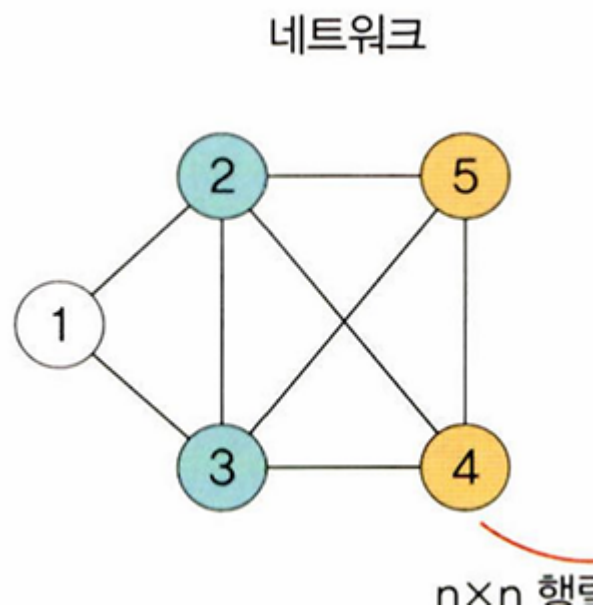
- 그래프 : 방향성이 있거나 없는 에지로 연결된 노드의 집합.



그래프 신경망(Graph Neural Network, GNN)

그래프 구조에서 사용하는 신경망.

- 그래프의 표현



1. 인접 행렬 (adjacency matrix) : 그래프를 컴퓨터가 이해하기 쉽게 $n \times n$ 행렬로 표현

인접 행렬 A_{ij}

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	1
3	1	1	0	1	1
4	0	1	1	0	1
5	0	1	1	1	0

2. 특성 행렬 (feature matrix)

각 입력 데이터에서 이용할 특성을 선택하고, 각 행이 선택된 특성에 대해 각 노드가 갖는 값을 의미

특성 행렬 $A + I$

		특성				
노드 1	1	1	1	0	0	
노드 2	1	1	1	1	1	
노드 3	1	1	1	1	1	
노드 4	0	1	1	1	1	
노드 5	0	1	1	1	1	

- $A + I$ (특성) 행렬은 인접 행렬 A 에 항등(단위)행렬 I 를 더해 자기 자신을 포함한 이웃 리스트를 만든 것. 어떤 이웃들의 특성을 반영할지 선택하는 역할을 $A+I$ 가 하게 됨.

특성 선택

- (예시) 노드 색상에 따른 원-핫 인코딩으로 **특성 행렬 X** 를 만들기.

$$X = \begin{bmatrix} 1 & 0 & 0 & \leftarrow \text{노드 1 (흰색)} \\ 0 & 1 & 0 & \leftarrow \text{노드 2 (파랑)} \\ 0 & 1 & 0 & \leftarrow \text{노드 3 (파랑)} \\ 0 & 0 & 1 & \leftarrow \text{노드 4 (노랑)} \\ 0 & 0 & 1 & \leftarrow \text{노드 5 (노랑)} \end{bmatrix}$$

- GCN 의 첫 레이어에서 다음 연산 수행

$$H^{(1)} = \sigma(\hat{A}XW)$$

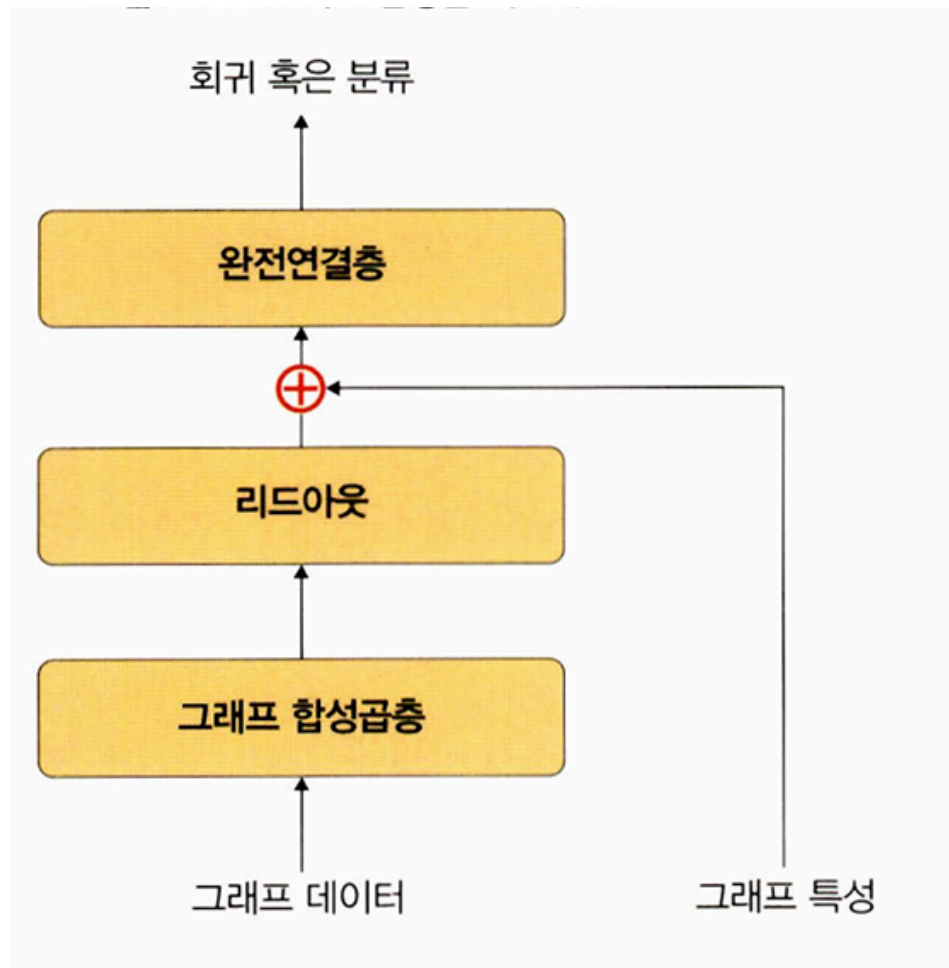
- $\hat{A}$: 정규화된 $A + I$
- X : 위에서 만든 특성 행렬
- W : 학습되는 가중치 행렬
- σ : ReLU 같은 활성화 함수

⇒ 그래프 구조($A+I$ 행렬, 누가 누구와 연결되어 있는지) 와 특성 행렬 X 를 조합해서 새로운 특성 표현(H) 을 학습하게 됨.

왜 $A+I$ 를 특성 행렬이라 표현했을까??

그래프 합성곱 네트워크(Graph Convolutional Network, GCN)

- 리드아웃(readout) : 특성 행렬을 하나의 벡터로 변환하는 함수. 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성



- SNS 관계 네트워크
- 학술 연구에서 인용 네트워크
- 3D Mesh

같은 분야에서 GCN 이 활발하게 활용된다.